CHAPTER 2

What the Automotive Industry Can Learn from Other Industries

In Chapter 1, we explored the vision for the modern automotive software realm, diving deep into the concept of a "smartphone habitat on wheels" and addressing the unique challenges and opportunities it introduces. We've demonstrated the imperative to innovate at a rapid pace and the complexities of ensuring functional safety in this hybrid landscape.

Now we shift our focus to the foundational concepts and technologies that underpin this vision. We will take a look at what made the smartphone and internet revolution successful and how we can apply these lessons to the SDV.

Learning from the Smartphone Folks: Standardization, Hardware Abstraction, and App Stores

Nokia's journey into the mobile world offers a significant lesson for the SDV. By 2009, Nokia had created a maze with 57 different and incompatible versions of its operating system. The consequences were devastating. For developers, it became a daunting task to create apps because of the vast fragmentation. For users, the resultant limited application ecosystem made Nokia's platform less attractive.

This tale mirrors a current challenge in the automotive world. Today, almost every car model, even those from a single manufac-

turer, employs custom hardware and software components sourced from various suppliers. The result: extreme fragmentation combined with monolithic programming frameworks, where creating a "vehicle app" that can run across multiple models of the same manufacturer seems nearly impossible.

However, the smartphone industry offers a blueprint for overcoming this fragmentation. Its solution was multipronged:

- *Standardized Application Programming Interfaces (APIs)* provide a consistent interface for developers, regardless of underlying system differences.
- An easy-to-use *programming language* and *app development environment* ensures that developers experience a flat learning curve and encapsulated environment for their applications.
- An *app store* acts as a centralized distribution hub, allowing third-party developers to get their applications seamlessly into the hands of end users.

So, what should the SDV industry learn from this? There are at least three lessons here:

1. *Unified APIs.* A set of standardized vehicle APIs would greatly simplify the process of creating software for vehicles. By ensuring that these APIs have minimal fragmentation, developers could write software once and have it work across multiple vehicle models. Based on industry trends, we believe that, in the long run, there will be a natural gravitation toward a select few dominant API sets, minimizing fragmentation across the industry. Within each OEM, we anticipate a more unified API approach, leading to consistency across their individual product lines.
2. *Hardware abstraction layer (HAL).* This acts as a bridge between the software applications and the multitude of vehicle hardware variations. It ensures that software can run irrespective of the underlying hardware differences, adding a layer of consistency and predictability.

3. *Supportive software stack (vehicle OS).* A robust software stack that's in harmony with the standardized APIs and HAL ensures that software can interact seamlessly with a vehicle's components, making software-driven innovations easier to introduce and adopt.

Figure 2-1 provides an overview of how best practices in the smartphone industry can be transferred to the automotive industry. The overview takes into consideration the fact that the automotive domain includes areas with requirements for real-time processing and functional safety.

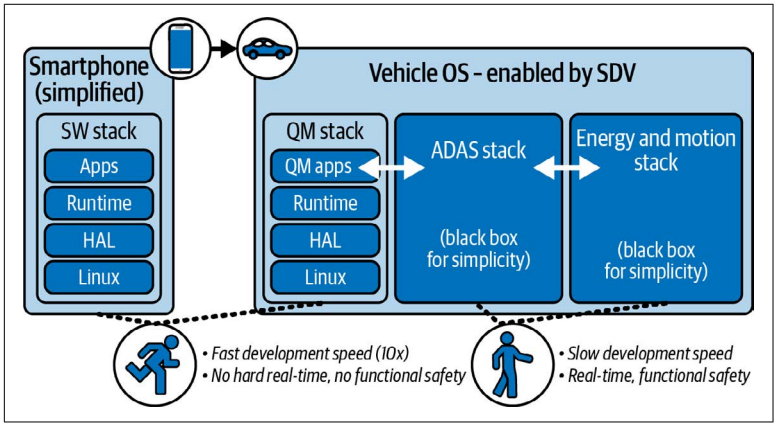

Figure 2-1. Applying smartphone best practices to automotive development

One key obstacle is the need for OEMs to differentiate themselves. Large OEMs in particular will want to keep large parts of their development proprietary while smaller OEMs might have a stronger interest in standardization and industry-wide reuse. The solution here can be to standardize frameworks and allow differentiation on the content side. For example, an OEM might want to standardize and open up the 70% to 80% of APIs that cover commodity functions. Using the same technical framework, this OEM could still manage the 20% to 30% of APIs that it wants to differentiate in a proprietary way.

Mohan B.V., Technology Head of Strategy, Mobility Next at Bosch BGSW, says:

> We are convinced that in the future, a lot of obligatory features will be cross-OEM. Does it make sense to build OEM-specific APIs? Seamless interoperability for data between vehicle, service provider, and driver is key, and harmonized API will enable this—this is what we have learned from the smartphone industry.

Can Yasin Gümüş, Senior Project Manager of Innovation Management at Bosch XC, adds:

> Defining standardized API opens the market for a community strategy. As an example, solving automated driving and driver assistance are huge challenges. One reason is the regional, complex problems the systems face. Opening up access to the solution space, developers around the globe can contribute to solving these kinds of problems. SDV opens a marketplace, allowing democratization of the development of automated driving and driver assistance.

Learning from the Internet Folks: Open Source and Cloud-Native Development

Even before the emergence of smartphones, the internet transformed the world by revolutionizing communication, access to information, commerce, and social interaction. Key enablers of this change include cloud-native technologies, the democratization of knowledge, and open source development. *Cloud native* refers to an approach to software architecture and development that leverages cloud-computing principles and services to enable fast adaptation to market demands, internet-level scalability, and resilience. Today, many of the software elements required for cloud native are supported by a large, global open source community.

As shown in Figure 2-2, key elements of cloud-native include microservices and APIs, containerization, continuous integration and delivery (CI/CD), and development and operations of IT systems (DevOps).

Microservices and APIs

Microservices are software components that encapsulate their data and business logic and make these available through well-defined APIs. Because microservice architectures are loosely coupled, they are ideally suited to support cross-organizational teams working on multiple microservices, evolving at different speeds.

Containerization

Containers provide the cloud-native runtime for microservices. Containers provide virtualization on the application level, which is more lightweight than the virtualization of an entire operating system. Containers are usually deployed on multiple network nodes in the cloud, providing scalability and resilience for the microservices running on them. They also provide additional levels of isolation, security, and systems management.

Continuous delivery and DevOps

Continuous delivery and DevOps are modern software development practices that ensure that the complexities, dynamics, and uncertainties of today's markets can be supported by frequent and reliable incremental code changes by cross-functional DevOps teams that collaborate throughout the product lifecycle and jointly take ownership of the deliverables.

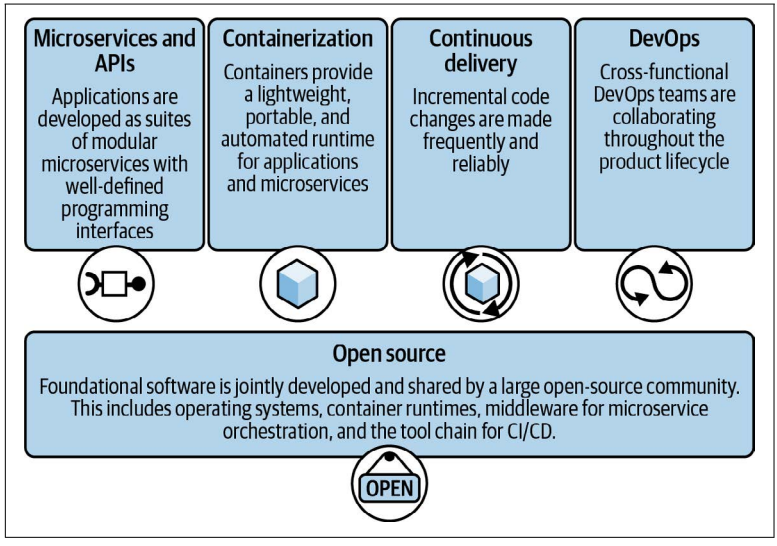

Figure 2-2. Key elements of cloud native and open source development

The success of the internet would not have been possible without *open source*, which has evolved from grassroots community projects to a mainstream movement. Today's thriving open source ecosystem has delivered a secure and scalable infrastructure, which is the backbone of the internet and most modern enterprise application landscapes. The open source community has delivered operating systems (e.g., Linux), container infrastructure (e.g., Kubernetes and Docker), middleware for microservices (e.g., Swagger), and the toolchain for CI/CD. Leading open source organizations in this space include the Linux Foundation, the Eclipse Foundation, and the Cloud Native Computing Foundation. The IT industry learned that by collaborating on non-differentiating parts like a Linux Kernel (shared by everything from smart TVs to cloud servers), more resources can be spent on the differentiating, customer-facing parts of their products.

Today, open source has evolved into a multi-billion-dollar market, including commercial enterprise-grade support, consulting, customizing, training, hosting services, dual licensing, and building commercial products on top of open source foundations.

In the SDV space, the Eclipse SDV working group is emerging as a highly visible open source initiative with a clear focus on SDV. Eclipse SDV aims to provide an open technology platform for the software-defined vehicle of the future and accelerating innovation of automotive software stacks through a vibrant open source community.